MODULE 34 · EXAM STRATEGY - ALL DOMAINS

Practice Exam 3

The last twenty. Same weighting, no repeats from either previous exam.

TYPE Practice exam QUESTIONS 20 PAUSE 9 seconds

admin@sthithapragna.com

Before you start

- **Nine seconds** per question. Pause if you need longer.
- **Commit out loud** before the walkthrough.
- This set leans slightly harder on diagnosis than the first two.
- Three questions turn on something being **impossible** rather than merely worse.
- Afterwards, average your three scores by domain. That is your real picture.

> ON THE EXAM
>
> Questions where an option is impossible rather than suboptimal are the ones worth slowing down on: a hard constraint eliminates faster than any judgement call.

A new serverless application needs Lambda functions to run SQL against an Aurora PostgreSQL cluster. The functions also call a third-party payment API over the internet on every request.

The data access layer is being written from scratch. The team wants no standing infrastructure cost while idle and short cold starts.

How should the functions reach the database?

A Attach the functions to the VPC and connect directly to the cluster endpoint

B Attach the functions to the VPC and connect through RDS Proxy

C Use the Aurora Data API over HTTPS, authorised with IAM

D Attach the functions to the VPC, add a NAT gateway, and use RDS Proxy

◆ PAUSE AND DECIDE

How should the functions reach the database?

A Direct VPC connection

Breaks the payment API call, because a VPC-attached function has no internet route without a NAT gateway. It also exhausts connections at concurrency.

B RDS Proxy in a VPC

Solves connections, still breaks the payment API, and adds a per-hour charge the team explicitly does not want while idle.

C Aurora Data API

No VPC, so internet access survives and cold starts stay short. No standing cost, and the code is being written from scratch anyway.

D RDS Proxy plus NAT gateway

Works fully, and adds two always-on billed components. It is the right answer to a question with different constraints.

THE SIGNAL

Three clauses each favour the Data API: an internet call, no idle cost, and code written from scratch. Option D is correct engineering for a different scenario.

> A DynamoDB table of 30 million orders is queried by a reporting job that filters on orderStatus and returns about 200 items. The job currently uses a Scan with a FilterExpression and consumes enormous read capacity.
>
> orderStatus has six possible values. The reporting job runs hourly.

What is the most efficient change?

A Create a GSI with orderStatus as the partition key and query it

B Add a FilterExpression on the Scan to reduce items returned

C Increase provisioned read capacity to absorb the Scan

D Enable DAX so repeated Scans are served from cache

◆ PAUSE AND DECIDE

What is the most efficient change?

A GSI on orderStatus

A Query on a GSI reads only matching items. Six values is low cardinality but acceptable for an hourly reporting job, and it replaces reading 30 million items with reading 200.

B Add a FilterExpression

It already has one. A filter is applied after items are read, so capacity is consumed on all 30 million regardless.

C More read capacity

Pays for the inefficiency rather than removing it. A Scan of 30 million items stays a Scan of 30 million items.

D DAX

Caches results, and the underlying Scan still runs on a cache miss. It also does not help an hourly job with changing data.

THE SIGNAL

The key fact is that FilterExpression runs after the read. Capacity is consumed on everything scanned, not on what is returned.

> A single-page application uploads files to S3 using presigned PUT URLs generated by an API. Testing the same URL with curl succeeds every time. From the browser, the request fails before it reaches S3, and the browser console reports a cross-origin error.
>
> The signature and expiry are confirmed correct.

What is missing?

A. The presigned URL must be regenerated with a longer expiry

B. The bucket must have Block Public Access disabled

C. The signing role needs additional S3 permissions

D. A CORS configuration on the bucket allowing the application's origin and the PUT method

◆ PAUSE AND DECIDE

What is missing?

A **Longer expiry**

The scenario confirms the expiry is correct, and curl succeeds with the same URL, which proves the signature is valid.

B **Disable Block Public Access**

A presigned URL carries the signer's permissions and is not public access. Block Public Access is not involved.

C **More permissions**

The same URL works in curl, which proves the signing role's permissions are sufficient.

D **A CORS configuration**

Works in curl but not in a browser is the CORS signature. The browser blocks the request itself, before S3 ever sees it.

THE SIGNAL

'Works in curl, fails in the browser' is diagnostic. Curl does not enforce CORS; the browser does, and the block happens client-side.

A developer adds request validation to an API Gateway REST API and changes an integration to a new Lambda alias. Both changes are saved and appear correctly when the method is reopened.

Calls to the production URL still reach the old alias and still accept invalid payloads. CloudWatch shows no errors of any kind.

What is the explanation?

A The API Gateway cache is serving stale responses

B The changes were saved but never deployed to the stage

C Request validation applies only to GET methods

D The Lambda alias needs a resource-based policy before it can be invoked

◆ PAUSE AND DECIDE

What is the explanation?

A **Stale cache**

Caching affects responses, not which alias is invoked, and it would not make invalid payloads pass validation.

B **Never deployed**

Configuration is saved against the API; a stage serves a deployment snapshot. Until a new deployment is created, the stage keeps serving the old one.

C **Validation is GET-only**

False. Body validation is most useful on POST and PUT, which are the methods carrying bodies.

D **Missing resource policy**

That produces visible errors and permission failures. The scenario states calls succeed against the old alias.

THE SIGNAL

Settings correct, behaviour unchanged, no errors anywhere. Nothing is broken - the change simply is not live.

> Clickstream events must be consumed by three independent systems: real-time personalisation, a fraud detector and an hourly warehouse loader. Each must see every event. After a bug, the team must be able to reprocess the last 24 hours from the beginning.
>
> Volume is around 20,000 events per second.

What should carry the events?

A One SQS standard queue polled by all three systems

B Three SQS queues, with the producer writing to each

C Amazon Kinesis Data Streams, with each system as a separate consumer

D An SQS FIFO queue to guarantee each system sees events in order

◆ PAUSE AND DECIDE

What should carry the events?

A One shared queue

A queue message is consumed once. Three consumers polling one queue means each event is processed by whichever gets there first.

B Three queues from the producer

Each system sees every event, and messages are deleted once consumed, so there is nothing to reprocess after a bug.

C Kinesis Data Streams

Records persist for the retention period, so multiple consumers each read the full stream and can replay from any point in the last 24 hours.

D FIFO queue

Adds ordering and caps at 3,000 messages per second with batching, well under 20,000. It also still deletes consumed messages.

THE SIGNAL

'Reprocess from the beginning' is the deciding clause. Queues delete on consumption; a stream retains, which is the whole difference.

> A telemetry pipeline runs 6,000 short workflows per second. Each runs four steps and finishes in under two seconds. Occasional duplicate processing is acceptable because the downstream store is idempotent.
>
> Cost at this volume is the primary concern.

Which Step Functions workflow type should be used?

A. A Standard workflow, one execution per event

B. A Standard workflow with a Distributed Map over batches

C. A Standard workflow with the callback pattern

D. An asynchronous Express workflow, one execution per event

◆ PAUSE AND DECIDE

Which Step Functions workflow type should be used?

A Standard per event

Standard is priced per state transition. Four transitions times 6,000 per second is an enormous bill for work finishing in two seconds.

B Distributed Map over batches

Distributed Map suits a large finite dataset, not a continuous stream. It adds orchestration overhead to a steady flow.

C Standard with callbacks

Callbacks are for waiting on an external decision. Nothing here waits, and it keeps the per-transition pricing.

D Asynchronous Express

Under five minutes, at-least-once is explicitly acceptable, and duration-based pricing is far cheaper at this volume. Exactly what Express is for.

THE SIGNAL

Every clause points at Express: under 5 minutes, duplicates acceptable, cost primary. 'At-least-once is fine' is the permission slip.

A monitoring vendor assumes a role in each of its customers' AWS accounts. It holds thousands of role ARNs. A review notes that if an attacker learns another customer's role ARN and persuades the vendor's platform to use it, the vendor's legitimate credentials would satisfy that trust policy too.

The vendor's platform runs from a fixed set of IP addresses.

What should each customer's trust policy require?

A. A condition on sts:ExternalId, with a unique secret issued per customer

B. A condition on aws:SourceIp restricting to the vendor's addresses

C. A condition on aws:MultiFactorAuthPresent

D. A permissions boundary on the role the vendor assumes

◆ PAUSE AND DECIDE

What should each customer's trust policy require?

A ExternalId per customer

Purpose-built for this. The vendor must present a secret only that customer issued, so knowing a role ARN alone is not enough.

B SourceIp restriction

The well-built distractor. The attack comes from the vendor's own platform at the vendor's own addresses, so an IP condition passes.

C Require MFA

MFA is for humans. An automated platform assuming roles thousands of times cannot present an MFA code.

D Permissions boundary

Caps what the role can do once assumed. It does nothing about who may assume it, which is the vulnerability.

THE SIGNAL

The mention of fixed IP addresses is planted to make B attractive. The attack originates from those very addresses, so the condition passes.

A mobile app authenticates with a Cognito user pool and calls a backend API behind an API Gateway Cognito authoriser. The API needs the caller's identity and group membership. The developer currently sends the ID token.

A security review objects that user attributes including email and date of birth reach a service that does not need them.

What should change?

A Remove the attributes from the user pool schema

B Send the access token instead, which carries scopes and cognito:groups

C Encrypt the ID token before transmission

D Switch to an identity pool so the API receives AWS credentials

◆ PAUSE AND DECIDE

What should change?

A Remove the attributes

The application genuinely needs email and date of birth. Deleting them solves the transmission by destroying the data.

B Send the access token

The access token carries scopes and cognito:groups, exactly what the API needs, without the user attributes it does not.

C Encrypt the token

It already travels over TLS. The objection is that the API receives the attributes at all, not that they are intercepted.

D Identity pool

Identity pools issue AWS credentials for calling AWS services directly. The app calls its own API, so a user pool alone is correct.

THE SIGNAL

A direction pair. The ID token says who the user is, for your app; the access token says what they may do, for an API.

An external auditor needs to decrypt a specific set of exported files for a three-week engagement. The files are encrypted with a customer managed KMS key whose policy is change-controlled and reviewed quarterly.

Access must be limited to decryption, revocable at any moment, and must not require modifying the key policy.

How should access be granted?

A Add a statement to the key policy and remove it after three weeks

B Share the plaintext data key with the auditor

C Create a KMS grant scoped to Decrypt, and retire it when the engagement ends

D Create a second KMS key and re-encrypt the files under it

◆ PAUSE AND DECIDE

How should access be granted?

A Edit the key policy

Directly contradicts the requirement, and puts a temporary change through a quarterly change-control process twice.

B Share the plaintext data key

Genuinely dangerous. It hands over something that cannot be revoked, cannot be audited, and protects every file that key encrypted.

C A KMS grant scoped to Decrypt

Grants exist for exactly this: temporary, operation-scoped access created and retired at runtime, with no key policy change.

D Second key and re-encrypt

Works, at the cost of re-encrypting the dataset and maintaining a second key. Disproportionate to the requirement.

THE SIGNAL

Three requirements - temporary, operation-scoped, no policy change - describe a grant precisely. That combination is the tell.

> An application on EC2 reads from S3 using access keys stored in a configuration file. A penetration test found that a server-side request forgery flaw in the application could be used to read instance metadata.
>
> Security wants long-lived credentials removed and the metadata attack surface closed.

What should be done?

A Move the access keys into AWS Secrets Manager and fetch them at startup

B Encrypt the configuration file at rest on the instance

C Rotate the access keys on a schedule with a Lambda function

D Attach an instance profile, delete the keys, and require IMDSv2 on the instance

◆ PAUSE AND DECIDE

What should be done?

A Keys in Secrets Manager

Circular. Reading Secrets Manager requires AWS credentials, so the instance still needs a stored credential to start.

B Encrypt the file

The application must decrypt it to use it, so anything running as the application can still read the keys.

C Scheduled rotation

Improves a bad pattern rather than replacing it, and does nothing about the metadata attack surface.

D Instance profile plus IMDSv2

The instance profile removes stored keys entirely, and IMDSv2 requires a PUT for a session token, which most SSRF attacks cannot perform.

THE SIGNAL

Two requirements, one answer. The instance profile addresses the credentials, IMDSv2 addresses the SSRF path to them.

A company has 300 project teams, each owning its own S3 buckets and DynamoDB tables. Each team must access only its own resources, and new teams appear weekly. Maintaining 300 near-identical IAM policies has become unmanageable.

All resources already carry a Project tag.

What should be implemented?

A One IAM group per team, with the existing policies attached to groups

B An ABAC policy comparing aws:PrincipalTag/Project to aws:ResourceTag/Project

C A permissions boundary per team applied to that team's roles

D A separate AWS account per team with cross-account roles

◆ PAUSE AND DECIDE

What should be implemented?

A Groups with the same policies

Improves how policies are attached, not how many exist. You still maintain 300 near-identical documents.

B ABAC on tags

One policy serves every team, and a new team needs only correct tags rather than a new policy. Exactly the problem described.

C A boundary per team

Boundaries cap, they do not grant, so you would still need the 300 granting policies. It adds 300 more documents.

D An account per team

Used at scale, and an enormous change adding 300 accounts to govern. Far beyond what the problem requires.

THE SIGNAL

'Already carry a Project tag' is the planted detail. ABAC needs disciplined tagging, and the scenario tells you it exists.

A CloudFormation stack creates an S3 bucket that an application fills with objects. Deleting the stack fails with DELETE_FAILED on the bucket. A separate requirement is to seed a DynamoDB table with reference data at creation time.

Both must be handled inside the template rather than by manual steps.

What should the template use?

A. A Lambda-backed custom resource that empties the bucket on Delete and seeds the table on Create

B. DeletionPolicy: Retain on the bucket, and a Mappings section for the seed data

C. A stack policy denying bucket deletion, and cfn-init for the seed data

D. Termination protection, and a DependsOn between the table and the application

◆ PAUSE AND DECIDE

What should the template use?

A Lambda-backed custom resource

The standard answer for both halves: emptying a bucket on Delete and running one-off setup on Create are things CloudFormation cannot do natively.

B Retain plus Mappings

Retain avoids the failure by orphaning a bucket that still exists and still costs money. Mappings hold literal values and cannot write to a table.

C Stack policy plus cfn-init

Stack policies govern updates, not deletion. cfn-init runs on EC2 instances, and there are none here.

D Termination protection plus DependsOn

Termination protection makes deletion harder rather than possible, and DependsOn only orders resources.

THE SIGNAL

Two requirements, one mechanism. 'CloudFormation cannot do this natively' is the custom resource signal, and emptying a bucket is its most cited example.

A pipeline deploys to staging automatically. Compliance requires a named release manager to authorise the production deployment explicitly, and to be notified rather than watching the console.

If nobody responds, the deployment must not proceed. The team does not want to build a custom approval application.

How should this be implemented?

A. A Lambda invoke action polling a DynamoDB approvals table

B. Disable the production stage and enable it manually per release

C. A stage condition checking a pipeline variable

D. A manual approval action in its own stage, publishing to an SNS topic

◆ PAUSE AND DECIDE

How should this be implemented?

A Lambda polling DynamoDB

Rebuilds manual approval by hand, including notification, timeout and audit trail. It is the custom application the team ruled out.

B Disable the stage

Not an approval, and it leaves no record of who authorised the release, which is what compliance actually asked for.

C Stage condition on a variable

Conditions evaluate automatically against rules. There is no human in the loop and no notification.

D Manual approval with SNS

The pipeline pauses, SNS notifies, PutApprovalResult permission names who may approve, and it fails after seven days if nobody responds.

THE SIGNAL

The compliance angle makes D right rather than merely convenient: the approval is recorded, attributed, and governed by an IAM permission.

> An Elastic Beanstalk environment runs a customer-facing API on twelve instances. Deployments must never reduce capacity, must never serve a mix of old and new versions, and must be reversible within seconds.
>
> The team accepts a longer deployment and a temporarily larger fleet.

Which deployment policy should be used?

A Rolling

B Rolling with additional batch

C Immutable

D All at once

◆ PAUSE AND DECIDE

Which deployment policy should be used?

A Rolling

Reduces capacity while batches update, and serves a mix of versions during the deploy. Fails two requirements.

B Rolling with additional batch

The closest wrong answer. It maintains capacity, and still serves a mix of versions mid-deploy, and rollback means deploying backwards.

C Immutable

New instances in a separate ASG, swapped only when healthy. Capacity maintained, no version mixing, and rollback is terminating the new instances.

D All at once

Takes the environment down during the deploy, failing the capacity requirement outright.

THE SIGNAL

Three requirements, and 'reversible within seconds' is what eliminates B. Rolling has already replaced the old instances; immutable has not.

A team uses the AWS CDK in TypeScript. They must set a newly released property on an S3 bucket that the L2 Bucket construct does not yet expose. CloudFormation supports the property fully.

They rely on the L2 construct's grant methods elsewhere in the stack and do not want to abandon it.

How should they set the property?

A Replace the L2 Bucket with a CfnBucket and set every property manually

B Access the underlying CfnBucket via node.defaultChild and use addPropertyOverride

C Define the bucket in a separate raw CloudFormation template

D Wait for a future CDK release to add support

◆ PAUSE AND DECIDE

How should they set the property?

A Drop to CfnBucket entirely

Works, and discards the defaults and grant methods the scenario says they want. An escape hatch keeps both.

B defaultChild plus addPropertyOverride

Exactly what escape hatches are for. The L2 construct is kept, and the property is written straight into the synthesised template.

C Separate raw template

Splits one bucket out of the stack that references it, creating a cross-stack dependency for a single property.

D Wait

Not a solution. Escape hatches exist so that CDK library lag never blocks you.

THE SIGNAL

'Do not want to abandon the L2 construct' separates B from A. Both set the property; only one keeps the grant methods.

A Lambda function reads an API endpoint from an environment variable. A developer updates the variable to a new endpoint and confirms it is saved in the console. Traffic continues going to the old endpoint.

The function is invoked through an alias named live, which points at version 12. The deployment pipeline was not run.

Why has the change not taken effect?

A The variable changed on $LATEST only. Version 12 retains the value captured at publish time

B Environment variable changes require the function to be restarted

C The alias caches configuration and must be recreated

D Environment variables cannot be changed after a version is published

◆ PAUSE AND DECIDE

Why has the change not taken effect?

A Changed on $LATEST only

A version freezes its environment variables at publish time. Editing them changes $LATEST, and version 12 is unaffected.

B Needs a restart

Lambda has no restart concept, and new execution environments start constantly. They would pick up a change if the version had one.

C Alias caching

An alias is a pointer, not a cache. It resolves to version 12, which genuinely holds the old value.

D Cannot change after publishing

Half right in a misleading way. You cannot change them on a published version, and you can on $LATEST, which is what happened.

THE SIGNAL

Option D states a true fact and draws the wrong conclusion. The console accepted the edit because it applied to $LATEST, not to version 12.

A company's CloudWatch bill exceeds its entire Lambda compute cost. Investigation finds debug-level logging enabled in production across 400 functions, and no log group with a retention period set.

The team must reduce cost without losing the ability to debug production issues when they arise.

What should they do?

A Delete the existing log groups and let them be recreated

B Move all log groups to the Infrequent Access class

C Set retention on every log group, and set the Lambda application log level to INFO

D Remove the debug logging statements from the code and redeploy

◆ PAUSE AND DECIDE

What should they do?

A **Delete the log groups**

Frees storage once and changes nothing about ingestion, so the cost returns. It also destroys the history.

B **Infrequent Access**

Reduces the ingestion rate and loses metric filters, subscription filters and alarms. The class also cannot be changed back.

C **Retention plus application log level**

Retention removes accumulated storage; the log level stops debug lines being ingested at all. Both are configuration, and the level can be raised again when debugging.

D **Remove the statements**

Requires redeploying 400 functions, and permanently removes the debug logging the team explicitly wants available.

THE SIGNAL

Option D is the near miss: it fixes the cost and destroys the capability the scenario says must be retained.

> An application runs as an ECS service on Fargate, instrumented with the X-Ray SDK. The identical code traces correctly when deployed as a Lambda function. On ECS no segments appear and the service is absent from the service map.
>
> The task role carries the AWSXRayDaemonWriteAccess policy.

What is missing?

A Active tracing must be enabled on the ECS service

B The task execution role needs the X-Ray permissions instead

C Fargate does not support X-Ray tracing

D The X-Ray daemon is not running as a sidecar container in the task definition

◆ PAUSE AND DECIDE

What is missing?

A **Enable active tracing**

Active tracing is a Lambda function setting. ECS services have no equivalent toggle, because collection is the daemon's job.

B **Task execution role**

The execution role starts the container; the task role is what application code uses. Writing segments is application code.

C **Fargate unsupported**

Fargate supports X-Ray fully, using a sidecar container exactly as EC2-launch-type tasks do.

D **No daemon sidecar**

The SDK sends segments to UDP port 2000 and something must listen there. On ECS that is a daemon sidecar in the task definition, which is absent.

THE SIGNAL

'The same code works on Lambda' is the clue: instrumentation is fine, so the difference is collection. Lambda collects for you; ECS does not.

> An API Gateway REST API integrates with a Lambda function calling a slow reporting database. Some requests return 504 Gateway Timeout. The function's timeout is 60 seconds and its Duration metric shows successful invocations taking up to 48 seconds.
>
> The function itself reports no errors.

What is happening?

A. API Gateway's 29-second integration timeout fires while the function is still running

B. The function is being throttled and never runs

C. The function returns a malformed proxy response

D. The database connection is being refused

◆ PAUSE AND DECIDE

What is happening?

A The 29-second gateway timeout

Invocations run up to 48 seconds, well past the 29-second integration timeout. API Gateway gives up and returns 504 while the function carries on.

B Throttling

Throttling returns 429, not 504, and the Duration metric shows the function running.

C Malformed response

A bad proxy response produces 502, not 504. The two codes mean different things.

D Connection refused

Would produce a function error in the logs. The scenario states the function reports no errors.

THE SIGNAL

The layered timeout problem: the function timeout of 60 seconds is longer than the gateway's 29, so the inner limit never takes effect.

> A DynamoDB table serving a retail catalogue is provisioned for the annual peak. For most of the year utilisation sits below 8%, and the team pays for peak capacity every month.
>
> Day-to-day traffic is highly predictable, with two known annual spikes.

What is the best change?

A. Switch permanently to on-demand mode

B. Enable provisioned auto scaling for the daily pattern, and buy reserved capacity for the baseline

C. Add a DAX cluster to absorb the peak

D. Reduce provisioned capacity to the average and accept throttling at the peak

◆ PAUSE AND DECIDE

What is the best change?

A On-demand permanently

The strongest distractor. On-demand suits unpredictable traffic, and this is described as highly predictable, where provisioned is substantially cheaper.

B Auto scaling plus reserved capacity

Auto scaling follows the predictable daily pattern, and reserved capacity discounts the baseline that is always there. It matches both stated facts.

C Add DAX

Reduces read capacity needs and adds a standing cluster cost. It does nothing about paying for peak write capacity all year.

D Average capacity, accept throttling

Saves money by failing customers at exactly the moment revenue is highest.

THE SIGNAL

'Highly predictable' is the deciding phrase, and it points away from on-demand. Predictability is what provisioned mode and commitments reward.

Scoring, and what comes next

- **17 or more across all three:** you are ready to book.
- **14 to 16 consistently:** passing territory, with a domain or two to firm up.
- **Below 14:** average your domain scores and revisit the weakest two modules.
- A domain weak in all three exams is a real gap. Weak in one is sampling noise.
- Sixty questions is a sample, not a simulation. The real exam has 65 and two minutes each.

> ON THE EXAM
>
> That is the end of the course. Module 31's rule is the one to carry into the exam room: answer the guide, know the gap, and do not let the second talk you out of the first.